Lab Sheet 6: MongoDB Basic commands

Student name : Madala Meghana Sri Lakshmi Iswarya

Reg. no : 23BCE7099

1. Use MongoDB to implement the following DB operations

1. Create a database called ‘vehicles’ and write a MongoDB query to select database as “vehicles”.

```
test> show dbs
admin   40.00 KiB
config  12.00 KiB
local   40.00 KiB
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin   40.00 KiB
config  60.00 KiB
local   40.00 KiB
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

```
vehicles> db.createCollection("two_wheelers", { capped: true, size: 50000 })
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport

red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) andprice.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Honda Shine",
| model: "gear",
| category: "125cc",
| colors_available: ["red","black","blue"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2022-05-01"),
| price: 80000
| },
| {
| bike_name: "TVS Jupiter",
| model: "gearless",
| category: "110cc",
| colors_available: ["white","black","grey"],
| manufacturer: "TVS",
| performance: 7,
| timestamp: new Date("2021-07-10"),
| price: 75000
| },
| {
| bike_name: "Yamaha R15",
| model: "gear",
| category: "150cc",
| colors_available: ["sport red","black"],
| manufacturer: "Yamaha",
| performance: 9,
| timestamp: new Date("2023-03-15"),
| price: 180000
| },
| {
| bike_name: "Bajaj Pulsar",
| model: "gear",
| category: "150cc",
| colors_available: ["black","blue"],
| manufacturer: "Bajaj",
| performance: 8,
| timestamp: new Date("2022-11-20"),
| price: 120000
| },
| {
| bike_name: "Suzuki Access",
| model: "gearless",
| category: "125cc",
| colors_available: ["red","silver"],
| manufacturer: "Suzuki",
| performance: 7,
| timestamp: new Date("2020-08-12"),
| price: 85000
| }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b16b4d5a22abc41d7c2907'),
    '1': ObjectId('69b16b4d5a22abc41d7c2908'),
    '2': ObjectId('69b16b4d5a22abc41d7c2909'),
    '3': ObjectId('69b16b4d5a22abc41d7c290a'),
    '4': ObjectId('69b16b4d5a22abc41d7c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Maruti Swift",
| model: "own",
| category: "car",
| variants: ["vxi","zxi","petrol"],
| manufacturer: "Maruti",
| performance: 8,
| timestamp: new Date("2022-06-12"),
| price: 700000
| },
| {
| vehicle_name: "Tata Bus",
| model: "commercial",
| category: "bus",
| variants: ["diesel"],
| manufacturer: "Tata",
| performance: 7,
| timestamp: new Date("2021-04-05"),
| price: 2500000
| },
| {
| vehicle_name: "Mahindra Bolero",
| model: "own",
| category: "car",
| variants: ["diesel"],
| manufacturer: "Mahindra",
| performance: 8,
| timestamp: new Date("2023-02-10"),
| price: 900000
| },
| {
| vehicle_name: "Ashok Leyland Truck",
| model: "commercial",
| category: "heavy truck",
| variants: ["diesel"],
| manufacturer: "Ashok Leyland",
| performance: 7,
| timestamp: new Date("2020-09-18"),
| price: 3000000
| },
| {
| vehicle_name: "Eicher Mini Truck",
| model: "commercial",
| category: "mini truck",
| variants: ["diesel"],
| manufacturer: "Eicher",
| performance: 6,
| timestamp: new Date("2019-12-01"),
| price: 1500000
| }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b16b565a22abc41d7c290c'),
    '1': ObjectId('69b16b565a22abc41d7c290d'),
    '2': ObjectId('69b16b565a22abc41d7c290e'),
    '3': ObjectId('69b16b565a22abc41d7c290f'),
    '4': ObjectId('69b16b565a22abc41d7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b16b4d5a22abc41d7c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2022-05-01T00:00:00.000Z'),
    price: 80000
  },
  {
    _id: ObjectId('69b16b4d5a22abc41d7c2908'),
    bike_name: 'TVS Jupiter',
    model: 'gearless',
    category: '110cc',
    colors_available: [ 'white', 'black', 'grey' ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: ISODate('2021-07-10T00:00:00.000Z'),
    price: 75000
  },
  {
    _id: ObjectId('69b16b4d5a22abc41d7c2909'),
    bike_name: 'Yamaha R15',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'sport red', 'black' ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: ISODate('2023-03-15T00:00:00.000Z'),
    price: 180000
  },
  {
    _id: ObjectId('69b16b4d5a22abc41d7c290a'),
    bike_name: 'Bajaj Pulsar',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 8,
    timestamp: ISODate('2022-11-20T00:00:00.000Z'),
    price: 120000
  },
  {
    _id: ObjectId('69b16b4d5a22abc41d7c290b'),
    bike_name: 'Suzuki Access',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'silver' ],
    manufacturer: 'Suzuki',
    performance: 7,
    timestamp: ISODate('2020-08-12T00:00:00.000Z'),
    price: 85000
  }
]
```

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b16b565a22abc41d7c290c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2022-06-12T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c290d'),
    vehicle_name: 'Tata Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2021-04-05T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c290e'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2023-02-10T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c290f'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2020-09-18T00:00:00.000Z'),
    price: 3000000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c2910'),
    vehicle_name: 'Eicher Mini Truck',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 6,
    timestamp: ISODate('2019-12-01T00:00:00.000Z'),
    price: 1500000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database.

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Honda Shine', price: 80000 },
  { bike_name: 'TVS Jupiter', price: 75000 },
  { bike_name: 'Yamaha R15', price: 180000 },
  { bike_name: 'Bajaj Pulsar', price: 120000 },
  { bike_name: 'Suzuki Access', price: 85000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Maruti Swift', price: 700000 },
  { vehicle_name: 'Tata Bus', price: 2500000 },
  { vehicle_name: 'Mahindra Bolero', price: 900000 },
  { vehicle_name: 'Ashok Leyland Truck', price: 3000000 },
  { vehicle_name: 'Eicher Mini Truck', price: 1500000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b16b4d5a22abc41d7c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2022-05-01T00:00:00.000Z'),
    price: 80000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b16b565a22abc41d7c290d'),
    vehicle_name: 'Tata Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2021-04-05T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c290e'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2023-02-10T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c290f'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2020-09-18T00:00:00.000Z'),
    price: 3000000
  },
  {
    _id: ObjectId('69b16b565a22abc41d7c2910'),
    vehicle_name: 'Eicher Mini Truck',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 6,
    timestamp: ISODate('2019-12-01T00:00:00.000Z'),
    price: 1500000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find(
| {performance: {$gt:5}},
| {bike_name:1, category:1, manufacturer:1, _id:0}
| )
|
[
  { bike_name: 'Honda Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'TVS Jupiter', category: '110cc', manufacturer: 'TVS' },
  { bike_name: 'Yamaha R15', category: '150cc', manufacturer: 'Yamaha' },
  { bike_name: 'Bajaj Pulsar', category: '150cc', manufacturer: 'Bajaj' },
  {
    bike_name: 'Suzuki Access',
    category: '125cc',
    manufacturer: 'Suzuki'
  }
]
vehicles> db.four_wheelers.find(
| {performance: {$gt:5}},
| {vehicle_name:1, category:1, manufacturer:1, _id:0}
| )
|
[
  {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  },
  { vehicle_name: 'Tata Bus', category: 'bus', manufacturer: 'Tata' },
  {
    vehicle_name: 'Mahindra Bolero',
    category: 'car',
    manufacturer: 'Mahindra'
  },
  {
    vehicle_name: 'Ashok Leyland Truck',
    category: 'heavy truck',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Eicher Mini Truck',
    category: 'mini truck',
    manufacturer: 'Eicher'
  }
]
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

```
test> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
admin      40.00 KiB
config     48.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
animal> db.createCollection("wild_animals",{capped:true,size:50000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
| {
| animal_name:"Lion",
| nature:"harm",
| favorite_foods:["meat","deer"],
| care_taker_name:"Ramesh",
| life_span:14,
| timestamp:new Date("2024-01-10"),
| expenses:50000
| },
| {
| animal_name:"Tiger",
| nature:"harm",
| favorite_foods:["meat","rabbit"],
| care_taker_name:"Suresh",
| life_span:16,
| timestamp:new Date("2023-05-15"),
| expenses:60000
| },
| {
| animal_name:"Elephant",
| nature:"harmless",
| favorite_foods:["grass","banana"],
| care_taker_name:"Mahesh",
| life_span:60,
| timestamp:new Date("2022-07-12"),
| expenses:80000
| },
| {
| animal_name:"Leopard",
| nature:"harm",
| favorite_foods:["deer","rabbit"],
| care_taker_name:"Ramesh",
| life_span:12,
| timestamp:new Date("2021-08-20"),
| expenses:45000
| },
| {
| animal_name:"Bear",
| nature:"harm",
| favorite_foods:["fish","honey"],
| care_taker_name:"Suresh",
| life_span:20,
| timestamp:new Date("2020-10-05"),
| expenses:55000
| }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b17a00fe26a488b77c2907'),
    '1': ObjectId('69b17a00fe26a488b77c2908'),
    '2': ObjectId('69b17a00fe26a488b77c2909'),
    '3': ObjectId('69b17a00fe26a488b77c290a'),
    '4': ObjectId('69b17a00fe26a488b77c290b')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female),favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name:"Cow",
| gender:"female",
| favorite_foods:["grass","hay"],
| animal_petname:"Gauri",
| life_span:20,
| timestamp:new Date("2024-02-01"),
| expenses:15000
| },
| {
| animal_name:"Dog",
| gender:"male",
| favorite_foods:["meat","biscuits"],
| animal_petname:"Tommy",
| life_span:13,
| timestamp:new Date("2023-04-10"),
| expenses:10000
| },
| {
| animal_name:"Cat",
| gender:"female",
| favorite_foods:["fish","milk"],
| animal_petname:"Kitty",
| life_span:15,
| timestamp:new Date("2022-06-20"),
| expenses:8000
| },
| {
| animal_name:"Goat",
| gender:"male",
| favorite_foods:["grass","leaves"],
| animal_petname:"Chotu",
| life_span:12,
| timestamp:new Date("2021-09-05"),
| expenses:7000
| },
| {
| animal_name:"Horse",
| gender:"male",
| favorite_foods:["grass","grains"],
| animal_petname:"Raja",
| life_span:25,
| timestamp:new Date("2020-11-12"),
| expenses:20000
| }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b17a08fe26a488b77c290c'),
    '1': ObjectId('69b17a08fe26a488b77c290d'),
    '2': ObjectId('69b17a08fe26a488b77c290e'),
    '3': ObjectId('69b17a08fe26a488b77c290f'),
    '4': ObjectId('69b17a08fe26a488b77c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b17a00fe26a488b77c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ramesh',
    life_span: 14,
    timestamp: ISODate('2024-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b17a00fe26a488b77c2908'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Suresh',
    life_span: 16,
    timestamp: ISODate('2023-05-15T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b17a00fe26a488b77c2909'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'banana' ],
    care_taker_name: 'Mahesh',
    life_span: 60,
    timestamp: ISODate('2022-07-12T00:00:00.000Z'),
    expenses: 80000
  },
  {
    _id: ObjectId('69b17a00fe26a488b77c290a'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'deer', 'rabbit' ],
    care_taker_name: 'Ramesh',
    life_span: 12,
    timestamp: ISODate('2021-08-20T00:00:00.000Z'),
    expenses: 45000
  },
  {
    _id: ObjectId('69b17a00fe26a488b77c290b'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Suresh',
    life_span: 20,
    timestamp: ISODate('2020-10-05T00:00:00.000Z'),
    expenses: 55000
  }
]
```

```
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b17a08fe26a488b77c290c'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'hay' ],
    animal_petname: 'Gauri',
    life_span: 20,
    timestamp: ISODate('2024-02-01T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69b17a08fe26a488b77c290d'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'biscuits' ],
    animal_petname: 'Tommy',
    life_span: 13,
    timestamp: ISODate('2023-04-10T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69b17a08fe26a488b77c290e'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: ISODate('2022-06-20T00:00:00.000Z'),
    expenses: 8000
  },
  {
    _id: ObjectId('69b17a08fe26a488b77c290f'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'grass', 'leaves' ],
    animal_petname: 'Chotu',
    life_span: 12,
    timestamp: ISODate('2021-09-05T00:00:00.000Z'),
    expenses: 7000
  },
  {
    _id: ObjectId('69b17a08fe26a488b77c2910'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'grass', 'grains' ],
    animal_petname: 'Raja',
    life_span: 25,
    timestamp: ISODate('2020-11-12T00:00:00.000Z'),
    expenses: 20000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database.

```
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 80000 },
  { animal_name: 'Leopard', expenses: 45000 },
  { animal_name: 'Bear', expenses: 55000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:15})
[
  {
    _id: ObjectId('69b17a08fe26a488b77c290e'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: ISODate('2022-06-20T00:00:00.000Z'),
    expenses: 8000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ramesh"})
[
  {
    _id: ObjectId('69b17a00fe26a488b77c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ramesh',
    life_span: 14,
    timestamp: ISODate('2024-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b17a00fe26a488b77c290a'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'deer', 'rabbit' ],
    care_taker_name: 'Ramesh',
    life_span: 12,
    timestamp: ISODate('2021-08-20T00:00:00.000Z'),
    expenses: 45000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
|
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'banana' ],
    expenses: 80000
  },
  {
    animal_name: 'Leopard',
    favorite_foods: [ 'deer', 'rabbit' ],
    expenses: 45000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'honey' ],
    expenses: 55000
  }
]
```

```
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
|
[
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'hay' ],
    expenses: 15000
  },
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'biscuits' ],
    expenses: 10000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 8000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 7000
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'grains' ],
    expenses: 20000
  }
]
```